

프로젝트 최종 보고서

Team 러닝머신

전자정보공학부 IT융합전공 20211513 김동준

전자정보공학부 IT융합전공 20211546 이채현

전자정보공학부 IT융합전공 20212996 허준범

프로젝트 최종보고서

팀명: 러닝머신

팀원: 20211513 김동준, 20211546 이채현, 20212996 허준범

[중간 발표 개선사항]

- Validation set에 증강 데이터가 섞여있는 문제 해결
- 증강 데이터가 섞여있는 t-SNE 결과가 혼재된 문제 해결
- 숫자 분류 모델 비교에서 OvO(One-vs-One) 기반 Linear Regression 모델 추가 비교

[YouTube link]

<https://youtu.be/5U9yKHSDtgM?si=hEA3jThtVbl0uvOj>

목 차

1. 서론.....	5
2. Background 전경색 분류 문제.....	7
3. 숫자 분류 문제.....	12
4. 숫자 분류 모델 비교 및 최종 선정.....	18
5. 성능 분석 및 데이터 증강.....	23
6. 하이퍼파라미터 조정.....	34
7. 최종 결론.....	39

1. 서론

1.1. 프로젝트 개요

프로젝트의 핵심은 **Colored MNIST** 데이터셋을 활용한 분류 모델을 구축하는 것이다.

우리가 사용할 데이터셋은 일반적인 흑백 **MNIST**와 달리, 숫자의 색상(전경색)과 배경 색상(배경색)이 서로 다른 조합으로 이루어져 있다는 특징이 있다. 즉, 단순한 흑백 이미지 처리를 넘어 색상 정보와 형태 정보를 모두 고려해야 하는 복합적인 데이터셋이다. 따라서 본 프로젝트에서는 이러한 데이터의 특성을 파악하고, 각 속성을 정확하게 판단할 수 있는 머신러닝 모델을 개발하는 것을 목표로 한다.

1.2. 데이터셋 개요

최종적인 학습 및 테스트를 위해, 제공된 **generate** 코드를 통해서 만들어진다. 원본 흑백 이미지를 가공하여 **Colored MNIST (Foreground-Background)** 데이터셋을 생성한다. 데이터의 생성 과정과 구조적 특징은 다음과 같다.



[그림 1] Colored MNIST

- 데이터셋 구축 과정

학습 데이터셋의 구축은 기존의 **Original MNIST(28×28, 단일 채널)**를 기반으로 진행되었다. 우리는 제공된 원본 흑백 이미지에서 **threshold**기반으로 정보를 추출하여 글자에 해당하는 정보를 만들고, 색상 정보를 주입하는 별도의 생성 코드를 활용하여, **Colored MNIST** 데이터셋을 직접 생성하였다. 결과적으로 모델 학습에는 원본 흑백 이미지가 아닌, 이 변환 과정을 거친 컬러 이미지가 입력 데이터로 사용된다.

- **Colored MNIST의 구조적 특징 (28×28×3)**

최종 생성된 **Colored MNIST**는 기본적으로 **color**로 확장되면서 **rgb** 정보를 담기

위해서 **rgb** 채널을 갖고있다. 따라서 **28x28x3** 사이즈의 3차원 데이터가 된다.

1.3. 모델 성능 평가 방법

- 성능 평가 방법 선정

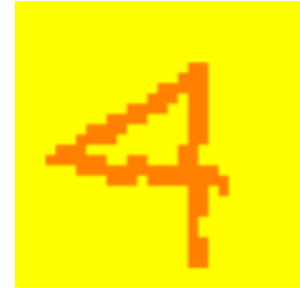
프로젝트에서는 글자를 보고 맞추는 것 목표이다. 이는 “위험 영상이냐 아니냐” 같은 추상적인 개념이 아니며, 명확하게 정답이 정해진 문제이다. 또한, 오분류를 집계에 포함하는 **Recall**이나 **Precision**은 순수한 정답을 파악하는데 방해가 될 수 있다. 따라서 본 프로젝트에서는 전체 데이터 중 올바르게 분류된 샘플의 비율을 의미하는 평가지표인 정확도(**Accuracy**)를 주요 기준으로 사용하였다.

또한, **MNIST**는 0~9의 클래스가 고르게 분포되어있기 때문에, 특별히 **data**의 **class** 비율 문제도 존재하지 않으므로, **F1 score**등에 사용되는 조화 평균 기법 또한 도입하지 않았다.

2. Background 전경색 분류 문제

2.1. 문제 정의

해당 문제는 입력된 **colored mnist image**를 글자의 색과 배경의 색으로 분류하는 문제이다. [그림 1]과 같은 입력이 들어온다면 배경색으로는 **Yellow**, 전경색으로 **orange**를 출력해야한다.



[그림 2] Colored MNIST

2.2. 색상 분류 접근 방식 도출

해당 문제는 사람은 큰 생각 없이 직관적으로 받아들이고 해석한다. 또한, 주어진 문제가 ‘분류’라는 점, 그리고 데이터 생성시 라벨을 기록했기 때문에 우리는 머신러닝 알고리즘 중 지도학습 방법인 분류 알고리즘 통해서 해결할 수 있는 문제라고 생각했다.

2.3. 머신러닝을 이용한 접근 방식

위 가설을 바탕으로, 머신러닝 알고리즘 예시로 몇 개를 사용하여 분류를 시도하였다. 선정 기준은 거리기반, 선형 분류기, 비선형 분류기 하나씩 하여 데이터 특징과 모델성능 사이의 차이를 보고자 하였다. (각 종류별 성능 비교 => 비선형이 적합하다 판단, 비선형에 근거하여 모델 선정의 논리적 과정) 머신러닝 **code** 별 수행 결과 비교는 다음과 같다.

정확도 종합	KNN	OVO (Linear Regression)	OVO(SVC)
전경색	0.7646	0.9043	0.9946
배경색	0.9999	0.8563	1.0000
Test 조건 : 특별한 tune없이 모든 hyper parameter는 auto로 cuML module을 통한 수행 Colored Mnist 60k 중 random selected 80%를 train set, 나머지를 validation set로 시행 Sklearn accuracy_score() 기준, 소수점 4자리에서 절사			

[표 1] 색 분류 에서의 모델 성능 비교

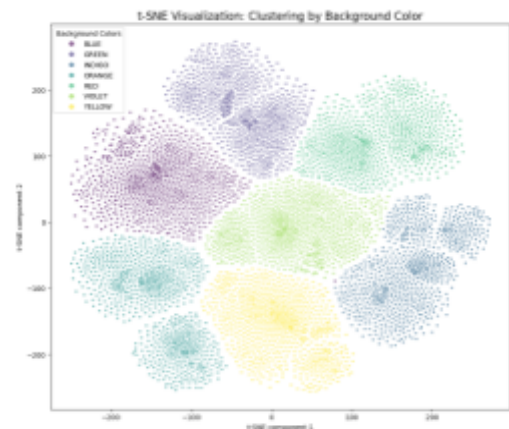
각 모델이 이러한 성능 차이를 보였을까? 일반 선형 분류기인 **Logistic regression**이나 거리 기반인 **KNN**보다 비선형 분류기인 **SVC**가 더 우수했는데, 이는 **MNIST**처럼 특징 공간에서 비선형성이 강한 이미지 데이터의 특성으로 인해 복잡한 패턴을 갖고 있기 때문일 것이다.

다만 가장 우수한 성능을 보인 **SVC** 역시 **100%**를 달성하지는 못했다. 하지만 사람이 볼 때는 배경색과 전경색 사이에 큰 난이도 차이를 느끼지 않는 문제이다. 하지만, 배경색은 **100%**에 가까운 정확도를 보이면서 전경색의 정확도가 떨어지다보니, 어떤 부분이 이를 어렵게 만드는지 분석하기 위해서 **t-SNE**를 수행했다.

2.4. t-SNE 기반 데이터 특성 분석

- 배경색 기준 t-SNE 시각화 결과

- 배경색 별로 매우 선명하게 분리된 클러스터가 형성됨을 확인.
- 이는 '배경색'라는 정보가 데이터를 구분하는 매우 강력한 특징으로 작용하고 있음을 의미.
- 결론적으로, 배경색 분류는 머신러닝 모델이 학습하기에 매우 용이한 형태의 분류 문제임을 알 수 있음.



[그림 3] background color 기준 t-SNE 수행 결과

- (개선사항) 전경색 기준 t-SNE 시각화 결과

- 전경색 기준으로 t-SNE를 실행한 결과가 배경색 기준 결과와 상이.
- 그래프에서 보듯, 하나의 큰 군집 내부에도 여러 색상이 혼재되어 명확히 분리되지 않는 경향을 보임.
- 이는 전경색 정보가 배경색에 비해 분류 특징으로서 명확성이 떨어짐을 의미.
- 숫자의 형태나 획의 위치 등 복합적인 요인에 따라, 색상 정보만으로는 구분이 어려운 영역이 존재하는 것으로 해석됨.

이는 실제 이미지 내에서 전경색 영역의 면적이 절반 이하로 작고, 일정하지 않은 모양(글자 모양)으로 퍼져 있기 때문일 것으로 추측됨.

따라서, 전경색 분류는 배경색 분류보다 더 어려운, 높은 난이도의 문제라는 가설, 그리고 비선형 모델에서 강세를 보이는 수행 결과와 일치하는 t-SNE의 직관적 분석을 알 수 있었다.

2.5. 팔레트 알고리즘 가정

이 성능을 끌어올리기 위한 방법을 고민하면서, 데이터에 대해서 더 본질적으로 고민했다. 여기서 우리는 이 문제를 풀 새로운 접근 방법을 생각해냈다.

주어진 **colored mnist** 데이터의 **generation code**를 보면 정해진 팔레트에서 생성되었다. 즉, “색상” 자체는 어떤 것이 나올지 라벨 값 즉 정답을 알고 있는 문제였던 것이다. 이 점을 이용해서 데이터의 픽셀들을 비교하면, 팔레트의 어떤 색에 대응하는지 파악할 수 있었다.

하지만, 이는 이미지에 쓰인 두 가지의 색을 찾기만 했을 뿐, 결국 어떤 색을 배경으로 사용할지에 대해서 추가적인 분석 데이터가 필요했다. 이미지를 보면, 무조건 배경이 차지하는 면적이 넓다는 것이 자명하다. 그러면, 이를 통해서 배경과 전경을 면적을 기준으로 접근해보고자 하였다. 이때, 이미지의 면적은 결국 픽셀 수와 같으므로, 팔레트의 각 색상에 대응하는 픽셀 수를 세어 기록하고, 가장 많은 수를 배경, 다른 하나를 전경으로 선택하는 알고리즘을 만들었다.

2.6. (개선사항) 알고리즘 실행 결과 분석

알고리즘의 성능 평가 결과는 다음과 같다.

정확도 종합	OVO(SVC)	알고리즘
전경색	0.9946	1.0000
배경색	1.0000	1.0000
OVO (SVC) Test 조건 : 특별한 tune없이 모든 hyper parameter는 auto로 cuML module을 통한 수행 Colored Mnist 60k 중 random selected 80%를 train set, 나머지를 validation set로 시행 Sklearn accuracy_score() 기준, 소수점 4자리에서 절사 알고리즘 테스트 조건 : 같은 validation set에 대해서 알고리즘 수행		

[표 2] 성능 비교

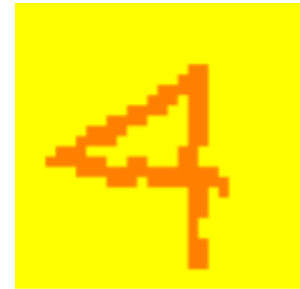
해당 알고리즘의 수행 결과는 배경색과 전경색 분류를 **validation set** 전체에서 모두 **100%**를 달성했다. 이는 직전 머신러닝 모델들에서 최고 수준을 달성했던 **SVC**와 비교해서도 두위에 있는 결과이다. 해당 알고리즘이 머신러닝 알고리즘들에 비해서 압도적인 정확도를 달성하였으며 추가적인 학습 과정도 필요하지 않았기에, 우리는 이 방법을 전경색, 배경색로 분류하는 솔루션으로 사용하기로 하였다.

추가적으로, 이 부분을 통해서 우리는 결정적인 규칙성을 찾아 알고리즘화 한다면 머신러닝에 비해서 성능과 효율성 면에서 우위에 있음을 알 수 있었다.

3. 숫자 분류 문제

3.1. 문제정의 및 필요

해당 문제는 RGB 색상 채널이 포함된 Colored MNIST 데이터셋을 입력 데이터로 활용하여 0부터 9까지 총 10개의 범주로 정의된 손글씨 숫자를 식별하는 다중 클래스 분류(Multi-class Classification) 모델을 구축하는 것을 목표로 한다. 처리 과정에서 모델은 이미지 내의 색상 정보와 형태적 특징을 종합적으로 분석 및 학습하여 최종적으로는 임의의 입력 이미지에 대해 가장 확률이 높은 클래스를 선정 함으로써 정답레이블을 오차 없이 판별하여 출력하는 것을 최종목표로 한다.



[그림 5] Colored MNIST

3.2. 데이터 특성 분석

생성된 데이터셋은 이미지파일과 각 샘플의 세부정보를 담은 데이터를 포함하고 있다. 라벨을 분석한 결과 데이터셋은 당증과 같은 4가지 핵심 속성으로 구성되어있음을 확인했다.

- 파일명(Filename)

각 이미지 데이터의 고유 식별자(identifier)역할을 하며, 데이터 로딩 및 관리의 기준이 된다.

- 숫자 레이블 (digit / Target Label)

0부터 9까지의 정수값으로 구성된 분류 모델의 예측 목표(Target)이다. 모델 학습 시 정답으로 사용된다.

- 전경색 (Foreground color)

숫자 획(Stroke)을 구성하는 색상 정보(예: Blue, Orange, Green 등)이다. 앞선 t-SNE 분석에서 확인했듯이, 숫자 형태와 결합되어 분류 난이도에 영향을 미치는 시각적 특징이다.

- **배경색 (Foreground color):**

숫자가 위치한 배경의 색상 정보(예: Violet, Yellow 등)이다. 전경색과의 대비를 형성하며, 모델이 학습 과정에서 전경과 배경을 분리해내는 능력을 요구하게 만드는 요소이다.

3.3. (개선사항) t-SNE 기반 원본 데이터 분석

- **색상 분포와의 유사성**

숫자를 기준으로 시각화했음에도 불구하고, 전체적인 군집의 형성 양상은 앞서 분석한 전경색기준의 분포와 매우 유사한 패턴을 보였다.

- **클래스 혼재**

0부터 9까지의 숫자 클래스들이 서로 독립적인 군집을 형성하지 못하고 있다. 시각화된 그래프에서 볼 수 있듯, 하나의 거대 군집 내부에도 서로 다른 숫자 레이블을 가진 데이터 포인트들이 무질서하게 혼재되어 있어 클래스 간 분리가 전혀 이루어지지 않은 상태이다.

데이터들이 선형적으로 분리되지 않고 복잡하게 얽혀 있는 현상이 관측되었다. 이에 따라 단순한 선형 분류기(Linear Regression)로는 해결이 불가능할 것이라 예측하였으며, 고차원의 비선형 패턴을 학습할 수 있는 SVC(Support Vector Classifier), KNN(k-Nearest neighbor model), Random Forest를 핵심 모델로 선정했다. (이에 대한 추가적인 분석 및 설명은 4단원에서 후술)

- **시행 결과 분석**

정확도	OVO (SVC)	OVO (LR)	KNN	Random Forest
숫자	0.8279	0.1757	0.8742	0.9260
Test 조건 : 특별한 tune없이 모든 hyper parameter는 auto로 cuML module을 통한 수행 Colored Mnist + colored font 60k 중 random 80%를 train set로 시행				

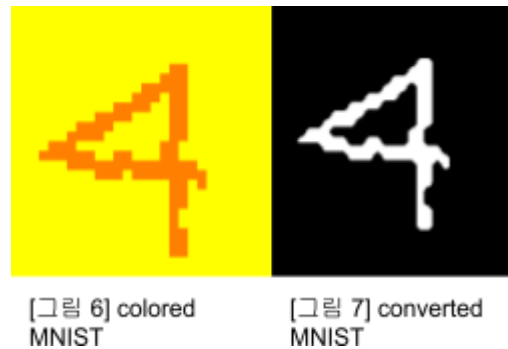
[표 3] 원본 데이터 성능 비교

3.4. (추가 아이디어)전처리 아이디어:

흑백 변환

- 전략 수립 배경

앞선 데이터 분석 결과, 원본 데이터의 색상 정보는 숫자 분류라는 본질적인 목표와 무관한 노이즈로 작용하며 모델의 학습 난이도를 불필요하게 높이는 것으로 확인되었다. 따라서 숫자의 형태적 특징에만 집중할 수 있도록, 모든 입력 이미지를 흑백으로 변환하는 전처리 과정을 핵심 전략으로 채택하였다.



- 변환의 기술적 근거

- 불필요한 정보(Noise) 및 편향 제거

숫자를 인식하는 데 있어 색상(Color)은 정보량을 제공하는 유의미한 변수가 아니라, 오히려 오버피팅(Overfitting)을 유발할 수 있는 불필요한 변수이다. 흑백 변환은 색상이라는 노이즈를 제거하고 숫자 형태라는 신호(Signal)만을 남김으로써 모델이 학습해야 할 특징 공간을 정제하는 효과를 가진다.

- 데이터 일관성 확보

원본 데이터에서는 같은 숫자 '1'이라도 빨간색 '1'과 파란색 '1'의 픽셀 값이 완전히 달라 모델이 이를 서로 다른 패턴으로 인식할 위험이 있다. 흑백 변환은 이러한 색상에 따른 픽셀 값의 분산을 제거하여, 모델이 동일한 숫자를 일관된 패턴으로 학습할 수 있도록 돕는다.

- 차원 축소

RGB 이미지를 그대로 사용할 경우 발생할 수 있는 '차원의 저주(Curse of Dimensionality)' 문제를 사전에 방지한다. RGB 이미지는 3개의 채널(Red, Green,

Blue)을 가지므로, 이를 1개의 채널(Gray)로 변환할 경우 입력 데이터의 차원이 1/3로 축소된다. 이는 연산 복잡도를 대폭 낮추어 학습 속도를 향상시킨다.

3.5. 흑백 이미지 t-sne 분석



[그림 8] digit 기준 t-SNE 수행 결과
- Colored

[그림 9] digit 기준 t-SNE 수행 결과
- BW converted

- **t-SNE 시각화를 통한 데이터 분포 변화 확인**

전처리 전략의 실효성을 검증하기 위해, RGB 색상 정보를 제거하고 흑백으로 변환된 데이터셋을 대상으로 다시 t-SNE 시각화를 수행하였다. 숫자 레이블을 기준으로 투영된 결과는 원본 데이터와 대비하여 다음과 같은 뚜렷한 구조적 변화를 보였다

- **군집 분리도의 획기적 개선**

원본 이미지 분석 시 관찰되었던 무질서한 혼재 양상과 달리, 흑백 변환 후에는 동일한 숫자 클래스끼리 강하게 응집하여 독립적인 군집을 형성하는 경향이 다수 관측되었다.

- **잔여 혼재 영역**

전반적인 개선에도 불구하고, 그래프의 중심부에서는 여전히 서로 다른 숫자 클래스들이 명확히 분리되지 않고 겹쳐 있는 고밀도 혼재 영역이 존재함

확인되었다.

- **분석 및 시사점 : 전처리의 유효성 입증**

흑백 변환이 데이터의 복잡도를 효과적으로 낮추었음을 시각적으로 확인하였다. 이는 앞서 수립했던 색상 정보는 분류를 방해하는 노이즈라는 가설이 타당했음을 입증하며, 흑백 변환이 숫자 분류 성능 향상을 위한 필수적이고 유효한 전처리 단계임을 보여준다.

3.6. (개선 사항)RGB vs BW 실제 정확도 비교

전처리 전후 성능 비교 (실험: 실제 흑백으로 성능 향상을 보일 수 있었음을 증명)

모델	원본	흑백 (배경 흑)	흑백 (배경 백)
OVO (SVC)	0.8279	0.9808	0.9809
OVO (LR)	0.1757	0.9501	0.1149
KNN	0.8742	0.9631	0.9635
Random Forest	0.9260	0.9779	0.9774
Test 조건 : 특별한 tune없이 모든 hyper parameter는 auto로 cuML module을 통한 수행 Sklearn accuracy_score() 기준, 소수점 4자리에서 절사			

[표 4] 전처리 전후 성능 비교

차원 축소를 통한 성능 향상 모든 모델에서 원본 컬러 데이터보다 흑백으로 전처리했을 때 성능이 유의미하게 향상되었다.

이는 숫자 판별 작업에서 색상(R, G, B) 정보가 불필요한 노이즈로 작용함을 시사한다. 흑백 변환을 통해 데이터의 차원을 축소하고, 오직 숫자의 형상정보만 남김으로써 모델의 학습 효율성을 극대화할 수 있었다.

하지만, 같은 흑백임에도 큰 정확도 차이를 보이는 모델이 존재한다. Linear Regression(LR) 모델의 결과를 보면 다음과 같다.

모델	흑백 (배경 흑)	흑백 (배경 백)
OVO (LR)	0.9501	0.1149

[표5] Linear Regression(LR) 모델 결과

선형 모델은 다음과 같이 $y = W \cdot x + b$ 형태를 띄는데, 각 픽셀 값에 가중치(w)를 곱하여 결과를 도출한다. 따라서 수학적으로 “연산” 하려면 중요한 특징이 큰 값이어야한다. 따라서 LR 모델은 흰색(255)이 중요한 특징으로 인식되기에 배경이 흰색인 경우에는 배경 자체를 강력한 신호로 오인하여 숫자를 전혀 인식하지 못했다. 반면, 배경을 0으로 만들면 수학적으로 배경이 계산에서 제외되므로 모델이 안정적으로 학습될 수 있다.

하지만, 비선형 모델은 이런 문제에 덜 민감하기 때문에, KNN이나 Random Forest 같은 모델은 배경색에 크게 영향을 받지 않기에 결과 값에서도 유의미한 차이를 보이지 않았다. 최종 선택한 OvO에서는 오차범위 이내라고 판단되는 수준의 성능 차이를 보였고, 다른 모델들에서 일관되게 성능이 높기 때문에, 배경을 검은색으로 하는 전처리 방식을 사용하기로 결정하였다.

4. 숫자 분류 모델 비교 및 최종 선정

4.1. 후보 모델 설명

KNN (K-Nearest Neighbors)

KNN은 거리 기반 분류 방식으로 작동하는 비모수 모델로서, 입력 데이터와 기존 학습 데이터 간의 유사도를 직접 계산하여 예측을 수행한다. 학습 과정이 거의 필요하지 않으며 구현이 매우 단순하다는 장점이 있으나, 예측 단계에서 고차원 데이터에 대해 상당한 연산 비용이 발생한다는 한계를 가진다. 그럼에도 불구하고 KNN은 MNIST 계열 문제에서 가장 기본이 되는 **baseline** 모델로 널리 활용되어 왔기 때문에, 본 프로젝트에서도 전처리 효과를 검증하고 다른 모델과의 성능 격차를 파악하기 위한 비교 기준으로 적합하다고 판단하여 후보 모델로 포함하였다.

Random Forest

Random Forest는 다수의 결정 트리를 생성하고 이들의 결과를 앙상블하는 방식으로 분류를 수행하는 모델이다. 단일 트리가 가지는 과적합 문제를 효과적으로 완화하면서도 안정적인 성능을 보여주기 때문에, 이미지처럼 패턴이 일정하지 않고 부분적으로 잡음이 존재하는 데이터에서도 비교적 우수한 예측력을 보인다. 특히 숫자 이미지의 윤곽이나 획의 방향성과 같은 비선형적인 특성을 개별 트리가 다양한 관점에서 학습함으로써, 단순한 거리 기반 모델보다 더 높은 표현력을 확보할 가능성이 있다. 이러한 안정성과 범용성을 고려하여 Random Forest는 본 프로젝트의 후보 모델로 선정되었다.

OvO(One-vs-One) 기반 Linear Regression

OvO 기반 Linear Regression은 다중 클래스 문제를 해결하기 위해 각 클래스 쌍마다 개별적인 회귀식을 학습시키는 방식으로 구성된다. 본 프로젝트에서는 모든 클래스 조합에 대해 선형 회귀 모형을 학습한 뒤, 예측 시 각 회귀 모델의 출력 오차를 비교해 최종 클래스를 결정하도록 구현하였다. MNIST 데이터는 비선형적 특성이 강해 일반적으로 비선형 모델이 더 유리하지만, 이러한 환경에서 선형 모델이 어느 정도의 분류력을 확보할 수 있는지를 확인하고, 전처리·특징 선택이 선형 성능에 미치는

영향을 검증하기 위해 **Linear Regression**을 후보에 포함하였다. 즉, **OvO** 기반 **Linear Regression**은 성능 경쟁보다는 선형 vs 비선형 모델의 성능 차이를 정량적으로 비교하기 위한 기준선(baseline) 모델로서 의의를 가진다.

OvO(One-vs-One) 기반 SVC (Support Vector Classifier)

OvO 방식 기반의 분류기는 다중 클래스 문제를 해결하기 위해 각 클래스 쌍마다 개별적인 이진 분류기를 학습시키는 전략으로 구성된다. 본 프로젝트에서는 **OvO** 방식을 **SVC(Support Vector Classifier)**에 적용하여 총 **45**개의 이진 분류기를 학습하는 구조를 사용하였다. **MNIST** 숫자 분류 문제는 클래스 간 경계가 선형적으로 분리되지 않고 복잡하게 얽혀 있는 특성을 보이는데, 이는 앞선 **t-SNE** 분석에서도 확인할 수 있었다. 이러한 데이터 구조는 비선형 경계를 학습할 수 있는 **SVC**의 장점과 **OvO**가 제공하는 미세한 클래스 간 분리 능력의 결합을 통해 효과적으로 다룰 수 있다. 또한 **GPU** 기반 **cuML** 라이브러리를 활용할 경우 **SVC**의 높은 계산 비용 역시 실질적으로 완화할 수 있어, **SVC**는 성능과 효율성 측면에서 최종 후보군으로 포함시키기에 충분한 타당성을 지닌다고 판단하였다.

4.2. 비교 기준

네 모델의 성능 비교는 완전한 공정성을 확보하기 위해 다음 기준 하에서 수행하였다.

- 모든 모델은 동일한 입력 데이터(흑백 이미지, **784**차원)를 사용하였으며, 전체 데이터셋을 **80%** 훈련, **20%** 벨리데이션선로 분할하였다.
- 모델의 성능 비교에 영향을 줄 수 있는 하이퍼파라미터 변화는 배제하기 위해 모든 모델은 기본(default) 파라미터 설정을 그대로 사용하였다.
- **GPU** 가속이 가능한 환경에서는 동일하게 **cuML** 라이브러리를 적용하여 연산 효율을 높였으며, 평가 단계에서는 **sklearn**의 **accuracy** 및 **weighted F1-score**를 공통적으로 적용하였다.

4.3. 모델별 성능 분석

모델	Accuracy	F1 Score (Weighted)
KNN	0.9631	0.9630
Random Forest	0.9779	0.9779
OvO (LR)	0.9499	0.9499
OvO (SVC)	0.9808	0.9808
Test 조건 : 특별한 tune없이 모든 hyper parameter는 auto로 cuML module을 통한 수행 Sklearn accuracy_score() 기준, 소수점 4자리에서 절사		

[표 6] 숫자 분류 모델 성능 비교 표

● KNN 성능 분석

KNN은 Accuracy 0.9631, weighted F1-score 0.9630를 기록하여 거리 기반 분류 모델로서 준수한 성능을 보였다. 그러나 KNN의 성능은 데이터의 분포 구조와 샘플 밀도에 크게 의존하며, 숫자 간 경계가 얽혀 있는 MNIST 특성(t-SNE 결과 참조)에서 거리 기반 분류가 충분히 강력하지 못하다. 연산량 측면에서도 데이터셋 규모가 커질수록 예측 시간이 급격히 증가하는 단점이 존재한다.

● Random Forest 성능 분석

Random Forest는 Accuracy 0.9779, weighted F1-score 0.9779로 KNN보다 높은 성능을 달성하였다. 이는 다수의 결정트리가 다양한 관점에서 데이터를 분류함으로써 안정적인 결과를 도출하기 때문이다. 그러나 이미지 형태 구조를 직접적으로 반영하기 어려운 트리 기반 모델의 특성상, 숫자 형태가 복잡하게 구성된 샘플들에 대해 최적의 분류를 수행하기에는 한계가 존재한다. 성능 면에서는 만족스럽지만, 모델이 학습하는 결정 경계의 표현력이 SVC 대비 낮다는

것이 모델 선택에 고려되었다.

- **OvO 기반 Linear Regression 성능 분석**

OvO 기반 Linear Regression은 Accuracy 0.9499, weighted F1-score 0.9499로 본 실험에서 가장 낮은 성능을 기록하였다. 이는 MNIST 숫자가 선형적으로 분리되기 어려운 분포를 가지고 있어, 선형 회귀 모델이 클래스 간 경계를 표현하는 데 한계가 있었기 때문이다. 또한 Random Forest와 SVC 대비 현저히 낮은 결과는 데이터가 비선형 모델에 더 적합하다는 점을 정량적으로 확인해주는 근거가 되었으며, 본 프로젝트에서 비선형 모델 선택의 타당성을 검증하는 기준선(baseline) 역할을 수행하였다.

- **OvO 기반 SVC 성능 분석**

OvO 기반 SVC는 Accuracy 0.9809, weighted F1-score 0.9809로 세 모델 중 가장 높은 성능을 기록하였다. SVC는 고차원·비선형 데이터에서 우수한 성능을 보이며, 이는 앞선 [그림 8]과 [그림 9] t-SNE 분석에서 숫자 데이터가 서로 겹치고 복잡하게 분포하는 특성과 일치하는 결과이다. 특히 RBF 커널을 통해 곡선적 경계를 형성할 수 있어, 서로 유사하게 보이는 숫자 클래스 간 미세한 패턴 차이를 효과적으로 학습할 수 있었다. 또한 GPU 기반 SVC 구현을 활용함으로써 학습 시간이 실용적인 수준으로 단축되어 성능·효율성 측면 모두에서 우수한 결과를 도출하였다.

4.4. 최종 모델 선정 및 이유

본 프로젝트는 위와 같은 성능 분석 및 데이터 구조적 특성에 기반하여 OvO 기반 SVC를 최종 숫자 분류 모델로 선정하였다. 선정 이유는 다음과 같다.

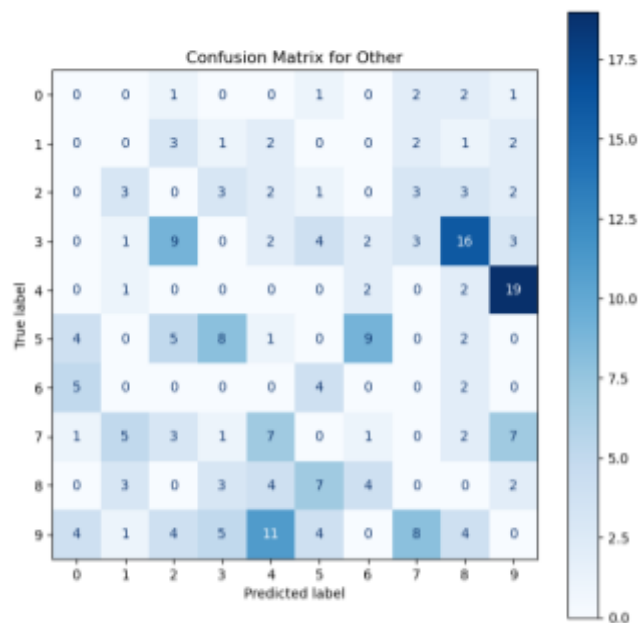
- SVC는 세 모델 중 가장 높은 분류 정확도(0.9809)와 F1-score(0.9809)를 기록하여 성능 면에서 명확한 우위를 보였다. 이는 단순히 오차율의 미세한 차이가 아니라, 숫자 형태의 구조적 다양성과 클래스 간 경계 복잡성을 고려했을 때 통계적으로 유의미한 차이이다.
- 앞선 [그림 8]과 [그림 9] t-SNE 시각화에서 나타난 바와 같이 MNIST 숫자 데이터는 단순한 선형 분리로는 구분이 어려운 구조적 복잡성을 지니고 있으며, 다수의 클래스가 중심부에서 혼재되어 있다. 실제 성능 비교에서도 OvO 기반 Linear Regression이 매우 낮은 정확도를 보인 반면, SVC는 비선형 경계를 학습함으로써 높은 분류 정확도를 기록하였다. 이는 비선형 모델이 본 데이터 특성과 구조적으로 더 잘 부합한다는 점을 실증적으로 확인한 결과라 할 수 있다.
- GPU 가속 환경에서 SVC의 연산 효율성이 확보되어 실제 모델 적용 시 과도한 계산 비용의 문제도 충분히 해소할 수 있었다. OvO 방식 특성상 분류기 수가 많음에도 불구하고, cuML 기반 병렬 처리가 가능하여 실용적 사용에 문제가 없었다.

이상의 분석을 종합하면, OvO 기반 SVC는 본 연구에서 고려한 전처리 환경(흑백 변환), 데이터 구조적 난이도, 성능 지표, 그리고 실행 효율성 측면에서 가장 균형 잡힌 모델로 판단되며, 이에 따라 최종 모델로 선정하였다.

5. 성능 분석 및 데이터 증강

위의 분석을 종합하여 선정된 OvO 기반 SVC의 성능을 강화하기 위해서, 성능 강화를 위하여 세부 분석을 진행하였다.

5.1. SVC-OVO 모델의 혼동 매트릭스 분석



[그림10] SVC-OVO 모델의 혼동 매트릭스 분석

SVC-OVO모델이 예측에 실패한 사례를 세밀하게 분석하기 위해 혼동 행렬을 시각화하였다. 위 표는 모델이 특정 숫자를 어떤 숫자로 착각했는지를 보여주며, 색상이 진할수록(파란색) 혼동 빈도가 높음을 의미한다. 분석 결과, 주로 형태학적 유사성이 높은 숫자 쌍에서 오분류가 집중되는 경향을 보였다.

- 실제 숫자 4를 9로 잘못 예측한 사례가 15건으로 가장 많았으며, 반대로 실제 숫자 9를 4로 예측한 사례도 11건이나 발생했다.
- 실제 숫자 3을 8로 잘못 예측한 사례가 14건으로 두 번째로 많았다.
- 실제 숫자 9를 7로 예측한 사례가 10건 발생하였다.

일반적인 혼동 행렬(Confusion Matrix)은 예측의 방향성을 구분하여 정밀도나 재현율을 분석하는 데 초점을 둔다. 그러나 모델이 겪는 형태학적 난이도를 직관적으로 파악하기 위해, 방향성을 제거하고 두 클래스 간의 오분류 총량을 합산한 대칭적 혼동 행렬을 더해 사용하였다. 이를 통해 예측의 쓸림 현상보다는, 숫자 쌍(Pair) 간의 구조적 유사성에 의한 혼동 그 자체를 정량화하였다. 15건 이상 발생한 종합 혼돈을 나열하면 다음과 같다.

- 실제 숫자 4를 9와 헷갈린 사례가 26건으로 가장 많았다.
- 숫자 3와 8을 잘못 예측한 사례가 16건으로 두 번째로 많았다.
- 숫자 9와 7를 잘못 판단한 사례가 17건 발생하였다.

따라서 3, 4, 7, 8, 9 클래스들을 중점으로 성능 개선이 필요하다고 판단했다.

5.2. 폰트 다양화

손글씨는 그 특성상 깔끔하지 못하고, 실제 사람이 육안으로도 명확히 구분되지 않는 것들이 많다. 그렇기에 모두 맞추는 것(human-above)은 불가능 하다고 판단, 지나치게 지저분한 글씨의 오류는 인정하고 깔끔한 글자를 더 잘 맞추는 것(human-like)이 성능을 높이는 방법이라고 생각했다.

5.2.1. 폰트 다양화 (Font Diversification)

폰트 증강을 도입, 모델이 특정 손글씨체의 픽셀 패턴에 과적합되는 것을 방지하고, 문자의 형태적 본질을 학습하도록 유도하여 글자를 맞추는 본질에 집중하기로 하였다.

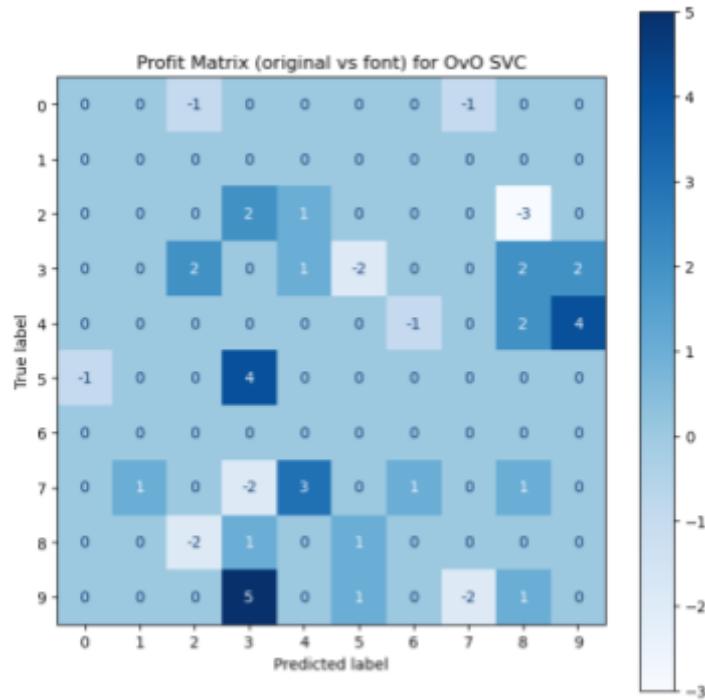
흔동이 도합 15건 이상 발생한 클래스 3, 4, 7, 8, 9에 대하여 MaruBuri 폰트로 생성된 이미지 각 6천장을 추가하였다.

- **특징 추출의 강건성 확보:** 획의 굵기, 뺄침(Serif), 기울기 등이 다른 다양한 폰트를 학습 데이터에 포함함으로써, 모델이 스타일적 요소가 아닌 글자의 고유한 구조적 특징에 가중치를 두도록 설계하였다.
- **클래스 내 분산(Intra-class Variance) 확대:** 동일한 숫자(클래스) 내에서도 다양한 형태적 변이를 주어, 추후 손글씨나 낯선 폰트가 입력되었을 때의 일반화(Generalization) 성능을 극대화하고자 하였다.



[그림11] Font로 생성된 MNIST 이미지

5.2.2. 증강 결과 분석



[그림 12] *이득 행렬

*이득 행렬 : 해당 행렬은 (폰트 증강 전 혼돈 행렬) - (폰트 증강 후 혼돈 행렬)를 나타낸다.

폰트 증강을 통해 기존 모델이 범하던 오류가 뚜렷하게 줄어든 구간은 다음과 같다.

항목	계
총 개선 (Total Profit)	+35
총 손실 (Total Loss)	-15
순수익 (Net Gain)	+20
*참고: 순수익(Net Gain)은 총 개선 건수(Total Profit)에서 총 손실 건수(Total Loss)를 차감하여 산출함 (+35 - 15 = +20). 결과: 폰트 증강 적용 시 일부 부작용(Side Effect)보다 긍정적인 교정 효과가 더 큼을 확인.	

[표 8] 정량 평가

Font 증강을 수행이후 전체적인 정확도의 개선이 있어 성능 향상을 보였다. 하지만 여전히 해당 클래스들이 15개 이상의 가장 높은 오류를 보였으며, 추가적인 방법으로 이를 보완해야한다 생각했다. Font 증강 이외의 새로운 방법을 찾기 위해서 오류가 난 이미지들을 육안으로 검사 하는 과정을 다시 진행했다.

5.3. 기울임 이미지 추가

Font 증강 이외의 새로운 방법을 찾기 위해서 오류가 난 이미지들을 육안으로 확인했고, 다수의 이미지들이 기울어 있음을 확인했다. 이들은 앞선 폰트 증강이 보충해주지 못한 데이터라고 판단하여, 추가적으로 기울임 증강 기법이 필요하다고 생각했다.

5.3.1. 회전 변환

MNIST 데이터셋의 혼돈 데이터 분석 과정에서 특정 숫자 클래스가 고유한 방향성 편향을 가지고 있음을 확인하였다. 이는 사람은 글씨를 쓸 때 팔꿈치나 손목을 회전축으로 고정하고 부채꼴 궤적을 그리며 작성하는 경향이 반영된 결과라고 볼 수 있다. 이러한 특성을 데이터셋에 인위적으로 주입함으로써, 모델이 정자체뿐만 아니라 '사람이 자연스럽게 쓴 기울어진 글씨'를 인식할 수 있는 능력을 부여하고자 하였다.

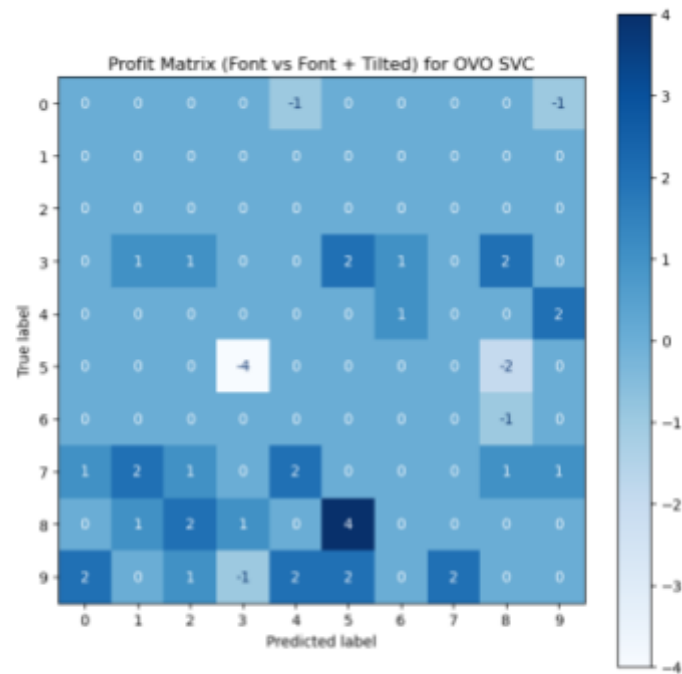
혼돈 data를 보면, 숫자 3은 주로 좌측으로, 8과 7은 주로 우측 5와 9는 우측으로 기울어진 형태가 지배적이다. 이러한 데이터의 불균형을 해소하고 모델의 강건성을 확보하기 위해 +5~ -5도의 회전 증강을 선택하였다. 각도를 무작위로 선택하여 폰트 증강 이후에도 가장 많이 틀리는 3, 4, 7, 8, 9 클래스에 대하여 6000장씩 선별하여 회전 증강을 수행했다.

[False] Lable = 8 & Prediction = 3



[그림13]

5.3.2. 증강 결과 분석



[그림14] tilt 증강 이후의 이득 행렬

- Font 증강 이후 대비 tilt의 이득

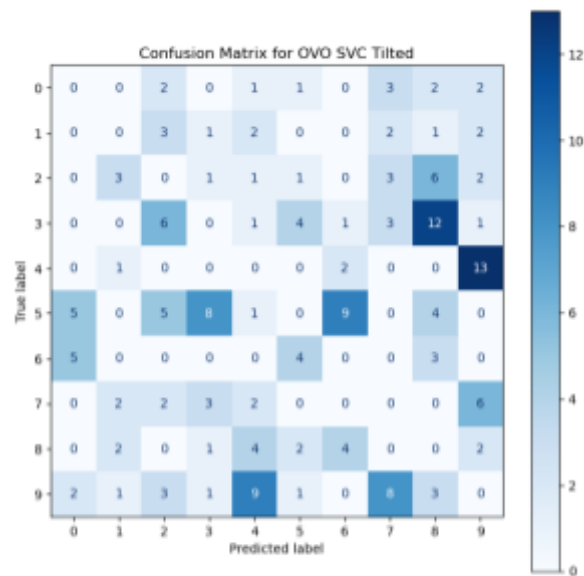
항목	손익
총 개선 (Total Profit)	+35
총 손실 (Total Loss)	-10
순수익 (Net Gain)	+25
*참고: 순수익(Net Gain)은 총 개선 건수(Total Profit)에서 총 손실 건수(Total Loss)를 차감하여 산출함 (+35 - 10 = +25). 결과: 틸트 증강 적용 시 일부 부작용(Side Effect)보다 긍정적인 교정 효과가 더 큼을 확인.	

[표 9] 종합 정량 평가

Tilt 증강은 **Font** 증강 대비 더 큰 이득이 있었다. 기존의 큰 **3-5** 클래스간의 손해가 있었지만, 전반적인 이득이 더 크므로 성공적이라 판단하고, 전체 혼동 행렬 분석을 수행하여 클래스별 성능 분석을 시행하였다.

5.4. 최종 증강 결과

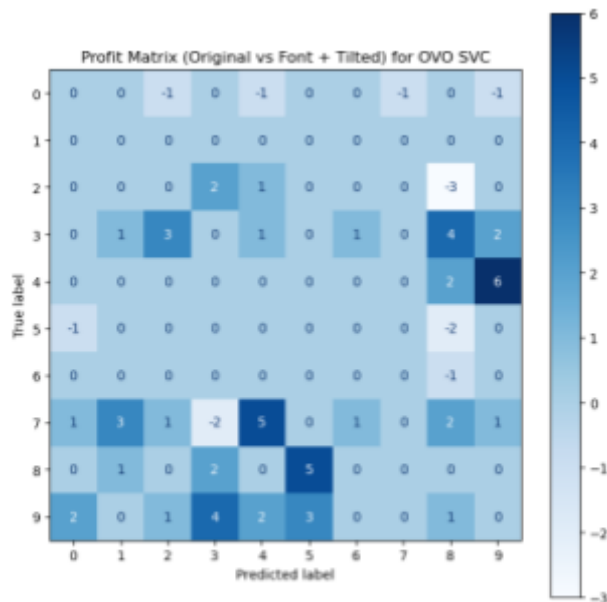
위에서 시행한 두 가지의 증강 기법을 도입해 얻은 결과의 최종 성능은 다음과 같다.



[그림15] 증강 이후의 혼동 행렬

최종 증강 이후 15개 이상의 혼돈 클래스를 없애고자 했던 최초 목표를 달성할 수 있었다.

증강으로 얻은 이익만을 편하게 분석하기 위하여, 증강 이전과 증강 이후의 이득 행렬을 표현하면 다음과 같다.



[그림 16] 최종 증강 이후의 이득 행렬

몇몇 클래스에서 손해가 있었지만, 전체 이득이 더욱 크므로 성공적으로 정확도가 올라갔으며, 특히 약점 보완을 시도한 클래스들에서 이득을 보고 있음을 확인할 수 있었다. 이를 통해서 각 단계별 선정된 증강 기법이 타당하고 실제 예측율에서의 이득이 발생했음을 확인할 수 있다.

위 각 증강 기법을 도입함으로써 얻은 정확도 이득을 정리하면 다음과 같다.

모델	단계별 손익	Accuracy	성능 증가분
증강 이전	-	0.9808	-
Font 증강	-15 / 35	0.9825	0.17%p
Font 증강 + Tilt 증강	-10 / 35	0.9846	0.21%p
이전 vs 최종 증강	-13 / 58	0.9846	0.38%p
*참고: Test 조건 : Colored Mnist 를 미리 전처리 기법을 수행하여 저장, 흑백 이미지 60k 중 random selected 80%를 train set, 나머지를 validation set로 시행			

[표 11] 증강 결과 비교

6. 하이퍼파라미터 조정

6.1. 탐색 대상 선정

OVO-SvC의 핵심 하이퍼 파라미터는 SVC모델의 파라미터로 결정된다. SVC의 핵심 하이퍼 파라미터 목록을 보면 다음과 같다.

```
class sklearn.svm.SVC(C, kernel, degree, gamma)
```

- 커널 함수

커널 함수와 관련하여 본 프로젝트에서는 RBF 커널에 집중하였다. 이는 앞선 모델별 성능 분석 결과에서 선형 분류 모델(OvO기반 Linear Regression)에 비해 비선형 모델(OvO기반 SVM)이 MNIST 데이터셋에서 일관되게 더 높은 분류 성능을 보였기 때문이다. 데이터 특성을 고려할 때, 선형 분류기인 Linear는 제외하고 RBF 커널을 선정하고 이를 기준으로 삼아 조정하였다.

RBF 커널을 사용하는 SVM 모델의 성능에 가장 큰 영향을 미치는 하이퍼파라미터는 정규화 계수 C 와 RBF 커널의 γ 값이다.

- 정규화 계수 C

모델이 오분류를 얼마나 허용할지를 결정하는 파라미터로, 값이 작을수록 마진을 넓게 유지하며 오분류를 더 허용하고, 값이 클수록 오분류를 강하게 억제하여 훈련 데이터에 민감해진다. 따라서 C 값이 지나치게 작을 경우 과소적합이 발생할 수 있으며, 반대로 너무 클 경우 과적합 위험이 증가할 수 있다.

- RBF 커널 파라미터 γ

하나의 데이터 샘플이 영향을 미치는 범위를 조절하는 역할을 한다. γ 값이 작을수록 결정 경계는 부드러워지며 전역적인 패턴을 학습하게 되고, 값이 클수록 결정 경계가 복잡해져 국소적인 패턴에 민감해진다.

6.1. 그리드 탐색

본 프로젝트에서는 SVM(RBF 커널)의 분류 성능을 향상시키기 위해 하이퍼파라미터 조정을 수행하였으며, 이를 위한 방법으로 그리드 서치(Grid Search)를 선택하였다. SVM은 정규화 계수 C 와 커널 파라미터 γ 의 값에 따라 모델의 결정 경계와 일반화 성능이 크게 달라지는 특성을 가지므로, 하이퍼파라미터의 체계적인 탐색이 필수적이다. 그리드 서치는 사전에 정의한 하이퍼파라미터 후보 집합에 대해 모든 조합을 순차적으로 탐색하는 방식으로, 탐색 과정이 명확하고 결과의 재현성이 높다는 장점이 있다.

특히 본 실험에서는 특정 하이퍼파라미터가 성능에 미치는 영향을 명확히 비교·분석할 필요가 있었기 때문에, 무작위 탐색(Random Search)보다 전수 탐색 방식인 그리드 탐색이 더 적합하다고 판단하였다. 보통의 경우 매우 큰 수행 시간으로 그리드 탐색의 선택이 부담이 크지만, 본 프로젝트에서는 GPU를 활용하는 cuML 라이브러리를 사용하므로 수행 시간을 줄여 그리드 탐색의 부담을 줄였다.

6.3. 그리드 탐색 과정

하이퍼파라미터 탐색은 효율성과 신뢰도를 단계적으로 높이는 2단계 전략으로 진행되었다. 먼저, 빠른 탐색과 VRAM 사용량 부담을 덜기 위해 최종 증강된 훈련 데이터에서 50%를 무작위로 추출하여 탐색용 데이터셋을 구성했다.

- 1차 탐색

모델의 기본값을 기준으로 지수적인 **scale**로 미리 **grid**를 형성하고, 분할된 탐색용 데이터셋으로 모델을 훈련시킨 후, 시작 전 미리 분리해 둔 단일 검증 세트로 성능을 평가하여 빠르게 유망한 파라미터 범위를 파악했다. 그 결과, **C=10**, **γ ='scale'** 조합이 가장 높은 정확도를 보였다. 이를 기반으로 근방의 다른 값들에 대하여 2차 세부 그리드 탐색을 시행하였다. 1차 탐색에서 나온 값을 기반으로 세밀한 2차 탐색 범위를 선정했다.

1차 그리드 탐색 범위	
C	1e-2, 1e-1, 1(초기값), 1e1, 1e2
γ	1e-9, 1e-8, 1e-7, *scale(초기값), 1e-6, 1e-5
*scale: (알려진 계산 공식을 이용하면 2.047e-07)	

[표 12] 1차 그리드 탐색 범위

- 2차 탐색

2차 탐색에 앞서, 간략하고 빠른 탐색이 중점이었던 1차 탐색과 다르게 2차 탐색은 훨씬 세부적인 튜닝을 목표로한다. 하지만 단일 검증 세트를 사용한 평가로 세부 튜닝을 진행하면 **validation set**에 대하여 과적합될 위험이 매우 크다. 따라서, 이러한 한계를 극복하고 더 신뢰도 높은 최적의 파라미터를 찾기 위해, 2차 탐색에서는 **5-Fold Cross Validation**을 도입했다. 2차 탐색 결과, **C=6**,

$\gamma=3e-07$ 조합이 가장 우수한 성능을 기록하여 최종 하이퍼파라미터로
 확정되었다.

2차 그리드 탐색 범위	
C	2, 4, 6, 8, 10, 12, 14, 16, 18
γ	1e-8, 3e-8, 5e-8, 7e-8, 9e-8, 1e-7, 3e-7, 5e-7, 7e-7, 9e-7

[표 13] 2차 그리드 탐색 범위

● 탐색 결과

모델	Accuracy	F1 Score
*1차 Grid Search Best	0.9818	0.9818
*2차 Grid Search Best	0.9681	-
*참고: 위 1차, 2차 그리드 탐색 과정은 서로 다른 데이터를 사용한 정확도를 나타낸것이므로 탐색 과정간의 직접적 성능비교는 불가능함. *2차 Grid Search간 validation set에대한 과적합 방지를 위하여 train set에서 cross validation기법 적용.		

[표 14] Grid Search에서 1차, 2차 모델 (Accuracy 및 F1 Score)

6.3. 최종 조정 결과

마지막으로, 교차 검증을 통해 선정된 최적의 하이퍼파라미터를 사용하여 모델을 생성하고, 증강이 완료된 전체 훈련 데이터셋을 이용해서 모델을 다시 학습시켜, 최종 모델의 성능을 최대화했다. 이를 적용해 달성한 성능은 다음과 같다.

모델	Accuracy	F1 Score
증강 기법 적용 이후	0.9846	0.9846
최종 모델	0.9882	0.9882
Test 조건 : Colored Mnist 를 미리 전처리 기법을 수행하여 저장, 흑백 이미지 60k 중 random selected 80%를 train set, 나머지를 validation set로 시행		

[표 15] 최종 모델 성능

7. 최종 결론

7.1. Svc 초기 성능 ~ 최종 성능 비교

Step	Accuracy	성능 증가분	비고
Base_Colored	0.8279	-	최초 실행
Base_BW	0.9808	15.29%p	흑백 전처리 도입
Font 증강	0.9825	00.17%p	증강 도입
Tilt 증강	0.9846	00.21%p	증강 도입
Hyper parameter tuned	0.9882	00.36%p	Grid Search 수행

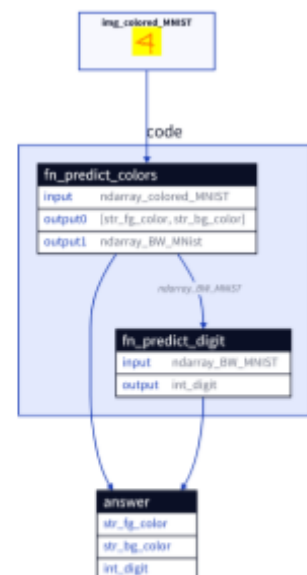
※모든 시행은 같은 validation set를 통해 Accuracy 측정

[표 16] 모델의 초기 성능 ~ 최종 성능 비교

정확도 98.82%, 전경색 및 배경색 인식 정확도 100%라는 매우 높은 결과를 확인할 수 있으며, 이는 색상 분리 및 숫자 분류 모델이 주어진 데이터에 대해 높은 성능을 보임을 알 수 있었다. 또한, 각 과정에서 성능이 증가함에 따라 적합한 성능 증가 기법을 적용했음을 알 수 있다.

7.2. 최종구조

해당 프로젝트의 최종 구조는 RGB 이미지를 입력받아 전경/배경색을 식별하고 흑백 이미지를 생성하는 1단계 전처리 과정, 그리고 이 이미지를 사전 훈련된 OVO-SVC 모델에 입력하여 숫자를 판별하는 2단계 추론 파이프라인으로 구성된다.



[그림17] 최종 구조

7.3. 최종 구조의 작동 검증

실제 추론

```
bw_images, predicted_fg, predicted_bg = predict_colors(test_images)
predicted_digit = predict_digit(bw_images)
```

[6]

```
... 100%|██████████| 10000/10000 [00:02<00:00, 4299.74it/s]
```

scoring

```
from sklearn.metrics import accuracy_score as sk_accuracy_score

digit_accuracy = sk_accuracy_score(test_labels, predicted_digit)
fg_accuracy = sk_accuracy_score(test_fg, predicted_fg)
bg_accuracy = sk_accuracy_score(test_bg, predicted_bg)

print(f"Digit Accuracy: {digit_accuracy * 100:.4f}%")
print(f"Foreground Color Accuracy: {fg_accuracy * 100:.4f}%")
print(f"Background Color Accuracy: {bg_accuracy * 100:.4f}%")
```

[7]

```
... Digit Accuracy: 99.7300%
      Foreground Color Accuracy: 100.0000%
      Background Color Accuracy: 100.0000%
```

Colored Mnist의 일부를 분할하여 가짜 **test set**을 생성하고, 이를 통합 파이프라인에서 수행하고 **scoring**하여 제대로 동작함을 확인했다.